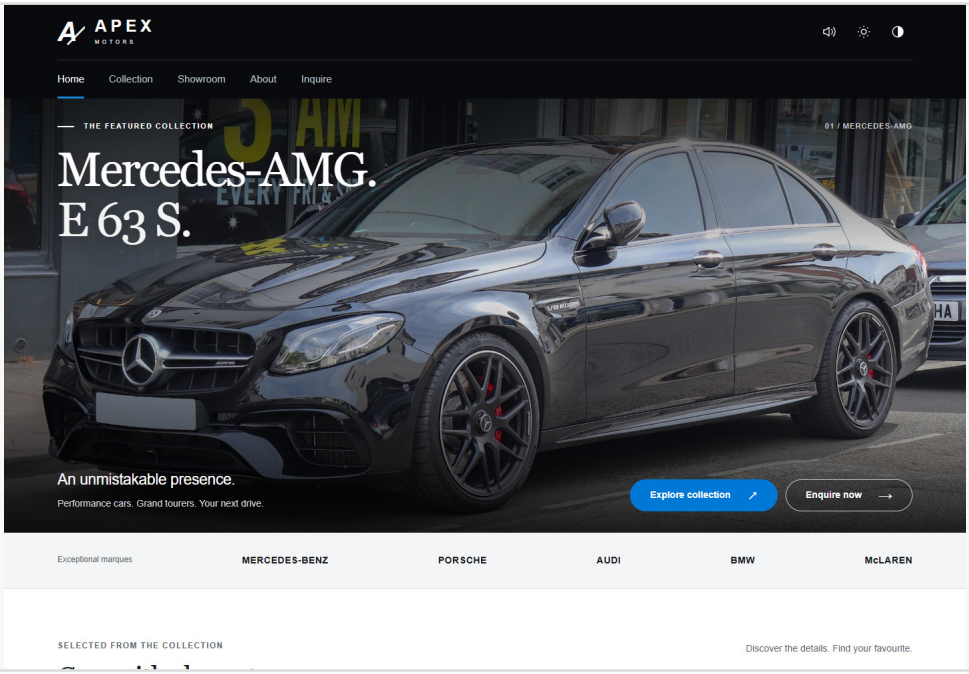
WEB TECHNOLOGIES | ASSIGNMENT 5

Advanced JavaScript and DOM Manipulation.

Apex Motors - 17 vehicles, high-resolution photography, an interactive showroom gallery, inquiry and team page.



Current photographic home page. Navigation opens independent HTML pages.

Team: Apex Motors **Group:** SE-2507
Members: Arsen, Kuttibay, Farabi, Muhammad
Prepared: 29 October 2026
Build: Five HTML pages, external CSS/JavaScript and local photographs, icons and sound.

Objective. Continue the team's automotive project with useful DOM updates, event handling, callbacks, data structures, sound and event-triggered animation. Replace section-only navigation with five connected pages while keeping shared appearance settings and a consistent experience.

Publication address: https://vertex-blip.github.io/Car-dealership-website/assignment5-dom/
Repository: https://github.com/Vertex-blip/Car-dealership-website
Status: Revised build checked locally; redeployment is pending unless confirmed by the accompanying deployment record.

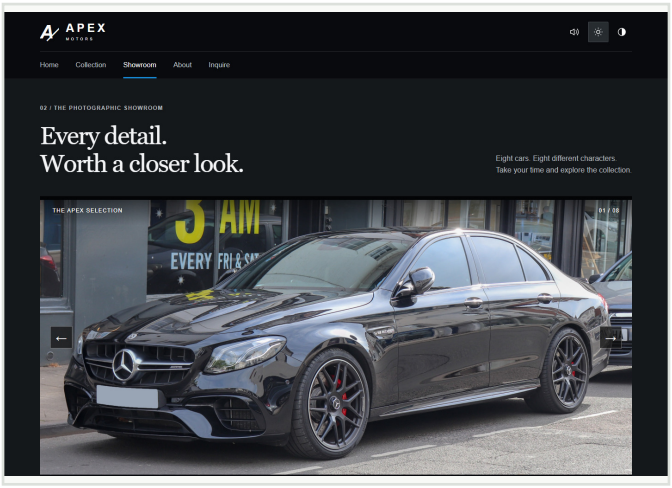
Day, night and changing color

The shared controls select individual and grouped DOM elements. Theme changes update the page appearance; the background control generates new hues rather than cycling three presets. This implements both alternatives in the styling task.

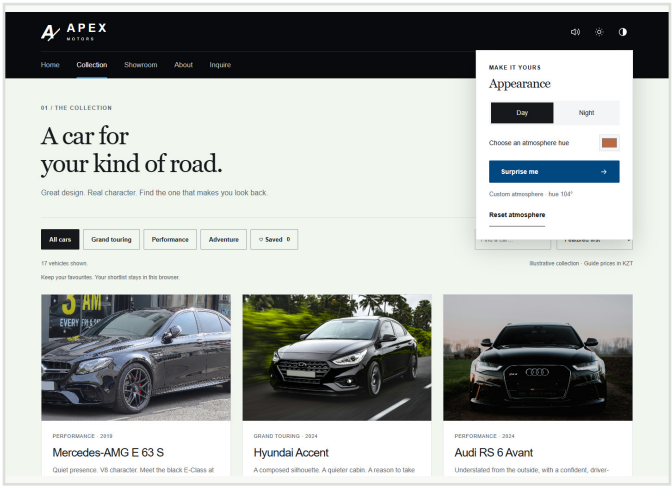
1. Select appearance controls and attach click listeners in the shared script.

2. Change theme attributes and CSS properties, then update button states and feedback.

3. Store preferences and restore them before rendering the next page.



Night appearance.



A generated background hue.

js/site.js

```
238    function applyAppearance() {
239      const dark = root.dataset.theme === "dark";
240      if (hue !== null) {
241        const color = `hsl(${hue} 28% ${dark ? "9%" : "95%"})`;
242        root.style.setProperty("--page-bg", color);
243        document.body.style.backgroundColor = color;
244      } else {
245        root.style.removeProperty("--page-bg");
```

js/site.js

```
274    function setTheme(theme) {
275      root.dataset.theme = theme;
276      write("apex-theme", theme);
277      applyAppearance();
278      Apex.playSound();
```

js/theme-init.js

```
  7    try {
  8      const saved = localStorage.getItem("apex-theme");
  9      if (saved === "dark" || saved === "light") theme = saved;
 10      const value = localStorage.getItem("apex-background-hue");
 11      if (value !== null && Number.isFinite(Number(value)))
```

Current source: js/site.js, js/theme-init.js

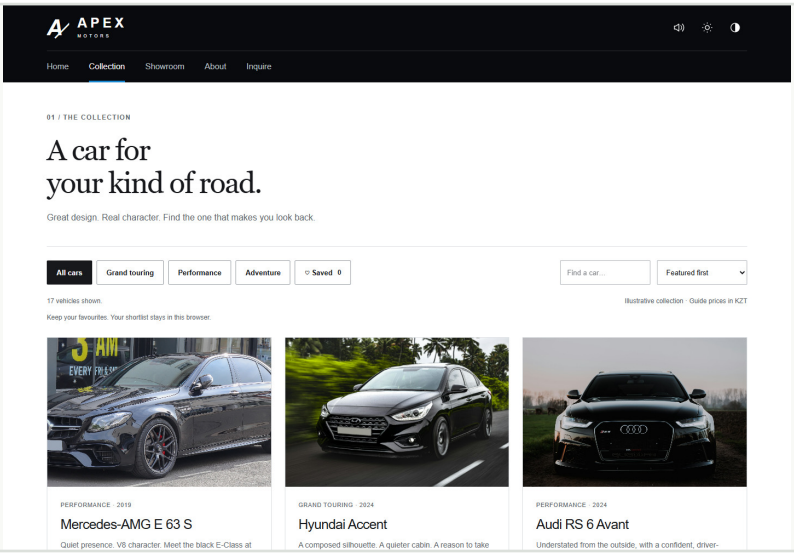
One collection, shared data

Seventeen vehicle objects provide the names, categories, photographs and descriptions used throughout the site. One array keeps cards and inquiry choices synchronized. The original images use their photographed vehicle names; prices and years are illustrative.

1. Create vehicle objects with properties and a reusable display method.

2. Iterate the collection explicitly to build cards and inquiry options.

3. Insert dynamic text into DOM elements and update the visible result count.



The 17-car collection, rendered from shared data.

js/data.js

```
24        alt,
25        description,
26        label() {
27          return `${this.categoryLabel} · ${this.year}`;
```

js/data.js

```
35    const vehicles = [
36      createVehicle(
37        "e63s",
```

js/inventory.js

```
183     let index = 0;
184     for (const vehicle of selectedVehicles)
185       fragment.append(makeCard(vehicle, index++));
186     if (!selectedVehicles.length)
187       fragment.append(element("p", "inventory-empty", emptyMessage()));
188     grid.replaceChildren(fragment);
```

js/booking.js

```
23    const vehicleOptions = document.createDocumentFragment();
24    for (const vehicle of window.Apex.vehicles) {
25      const option = document.createElement("option");
26      option.value = vehicle.id;
27      option.textContent = vehicle.name;
```

Current source: js/data.js, js/inventory.js, js/booking.js

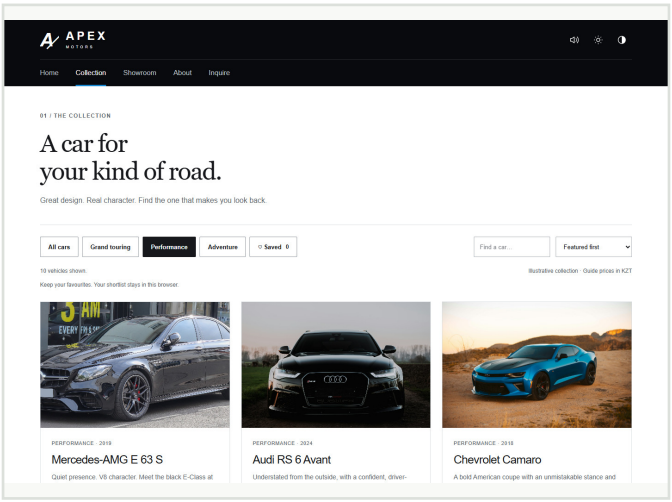
Filter and save a collection

Category buttons apply user input to a switch statement and higher-order filtering. Saved vehicles extend the same collection without maintaining another set of cars.

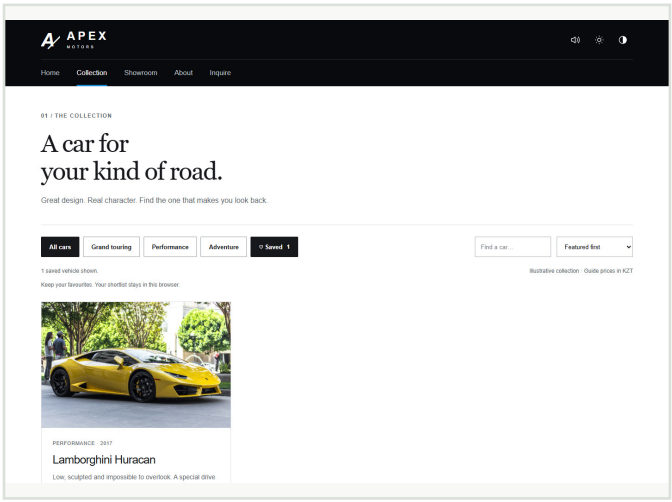
1. Read the selected category and choose its condition with switch.

2. Pass the matching function to filter and render the returned collection.

3. Save vehicle IDs, update control states and restore the shortlist when storage is available.



Category-filtered collection.



Saved vehicles selected.

js/inventory.js

```
65    function matchesCategory(vehicle) {
66      // The switch determines which objects reach the visible card grid.
67      switch (category) {
68        case "touring":
69          return vehicle.category === "touring";
70        case "performance":
71          return vehicle.category === "performance";
72        case "adventure":
73          return vehicle.category === "adventure";
74        default:
75          return true;
```

js/inventory.js

```
164     // filter is higher-order: each named predicate is passed as a callback.
165     const selectedVehicles = vehicles
166       .filter(matchesCategory)
167       .filter(matchesSearch)
168       .filter((vehicle) => !savedOnly || savedIds.includes(vehicle.id));
169     switch (sortSelect?.value) {
170       case "price-low":
171         selectedVehicles.sort((a, b) => a.price - b.price);
```

Current source: js/inventory.js

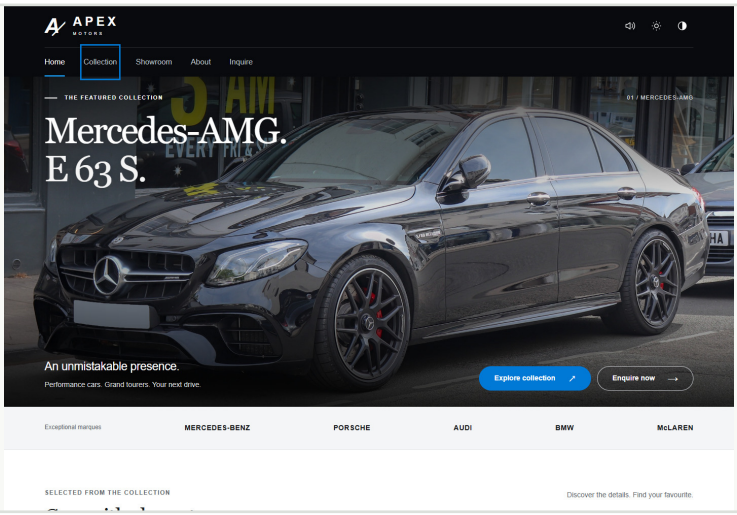
Navigation you can hear

The five navigation links open different HTML pages. Arrow keys move focus through visible navigation controls with a short sound; Enter activates the focused link.

1. Register click and keydown listeners; ignore text-entry fields for page-level arrows.

2. Use the direction to calculate the next index with wraparound.

3. Focus the next control without scrolling, then play the navigation sound.



Visible keyboard focus on the shared navigation.

js/site.js

```
123    const nav = document.getElementById("assignmentNav");
124    const navToggle = document.getElementById("navToggle");
125    const links = nav ? [...nav.querySelectorAll(".nav-link")] : [];
```

js/site.js

```
163      let next;
164      switch (event.key) {
165        case "ArrowRight":
166        case "ArrowDown":
167          next = (index + 1) % links.length;
168          break;
169        case "ArrowLeft":
170        case "ArrowUp":
171          next = (index - 1 + links.length) % links.length;
172          break;
173        case "Home":
```

js/site.js

```
187      links[next].focus({ preventScroll: true });
188      Apex.playSound();
189    });
190    document.addEventListener("click", (event) => {
```

Current source: js/site.js

05 | CALLBACK INTERACTION

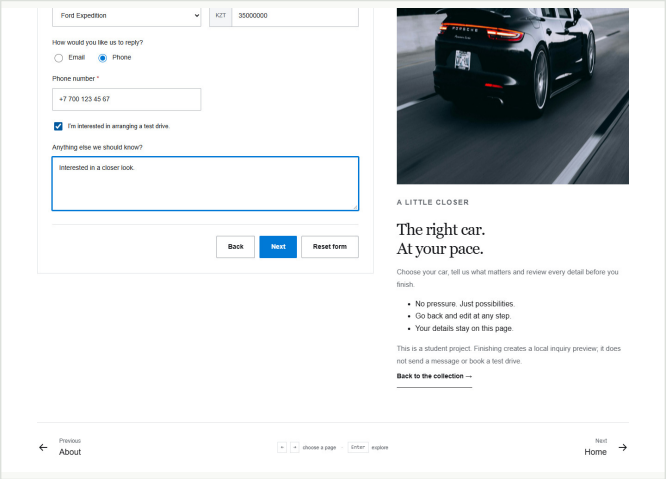
One inquiry, three steps

The selected callback alternative is a multistep form. Contact, Preferences and Review reuse the same inputs, so moving Back does not erase entered values.

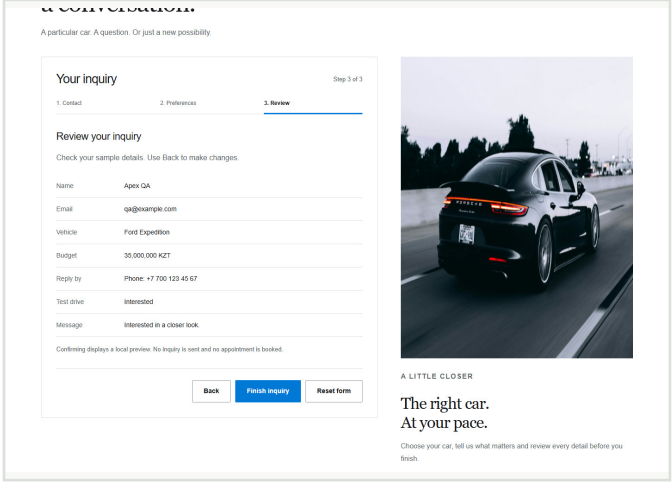
1. Track the current step and validate it before a forward move.

2. Pass the step-display function as a callback to the navigation function.

3. Display the requested fieldset while retaining existing input state.



Preferences step.



Review before confirmation.

js/booking.js

```
189    function navigateStep(direction, onStepChange) {
190      if (direction > 0 && !validateStep(currentStep)) {
191        status.textContent =
192          "Please correct the highlighted fields before continuing.";
193        return;
194      }
195      const nextStep = Math.max(0, Math.min(2, currentStep + direction));
196      onStepChange(nextStep);
197    }
198    nextButton.addEventListener("click", () => navigateStep(1, showStep));
```

js/booking.js

```
197    }
198    nextButton.addEventListener("click", () => navigateStep(1, showStep));
199    previousButton.addEventListener("click", () => navigateStep(-1, showStep));
200
```

js/booking.js

```
156    function showStep(index, moveFocus = true) {
157      currentStep = index;
158      steps.forEach((step, position) => {
159        step.hidden = position !== currentStep;
```

Current source: js/booking.js

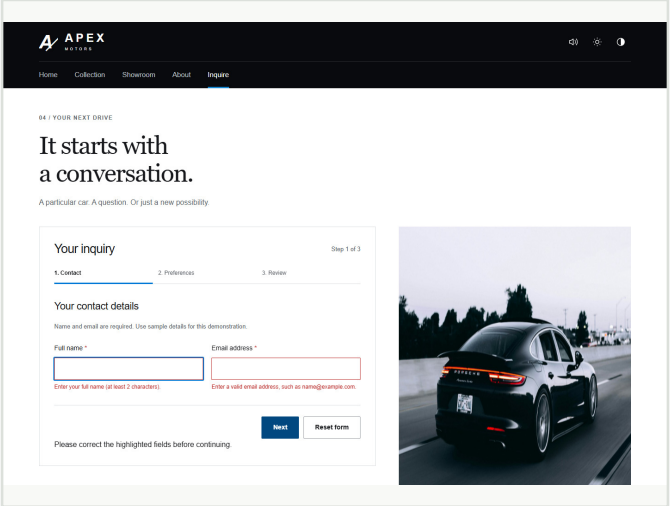
Validate, review and reset

The inquiry checks the current step, displays understandable errors and produces a local confirmation. It is a classroom preview and does not POST personal details.

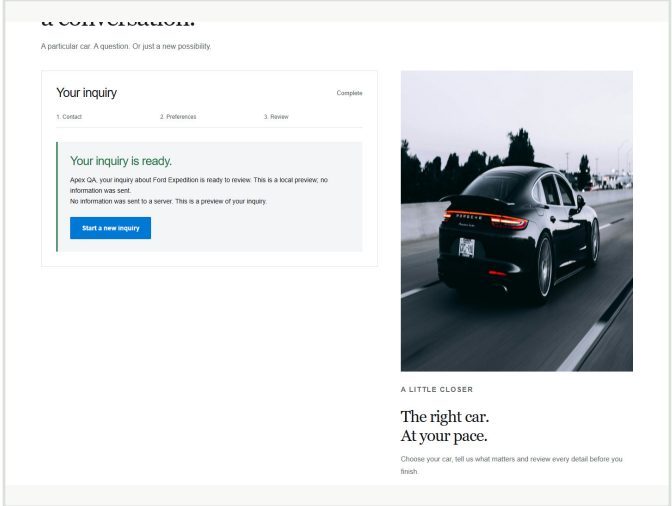
1. Prevent default submission, show errors and focus the first invalid control.

2. Build the review from current values, then show confirmation without reloading.

3. Reset inputs, error messages and step state for a new inquiry.



Field validation feedback.



Local confirmation.

js/booking.js

```
 98    function validateStep(index, focusFirst = true) {
 99      let firstInvalid = null;
100      for (const id of fieldsByStep[index]) {
101        if (!setError(id, fieldError(id)) && !firstInvalid)
102          firstInvalid = document.getElementById(id);
103      }
104      if (focusFirst && firstInvalid) firstInvalid.focus();
105      return !firstInvalid;
106    }
107
```

js/booking.js

```
201    function clearValues() {
202      form.reset();
203      form.querySelectorAll("input, textarea").forEach((input) => {
204        if (input.type === "radio") input.checked = input.id === "replyEmail";
205        else if (input.type === "checkbox") input.checked = false;
206        else input.value = "";
207      });
208      vehicleSelect.value = "";
```

Current source: js/booking.js

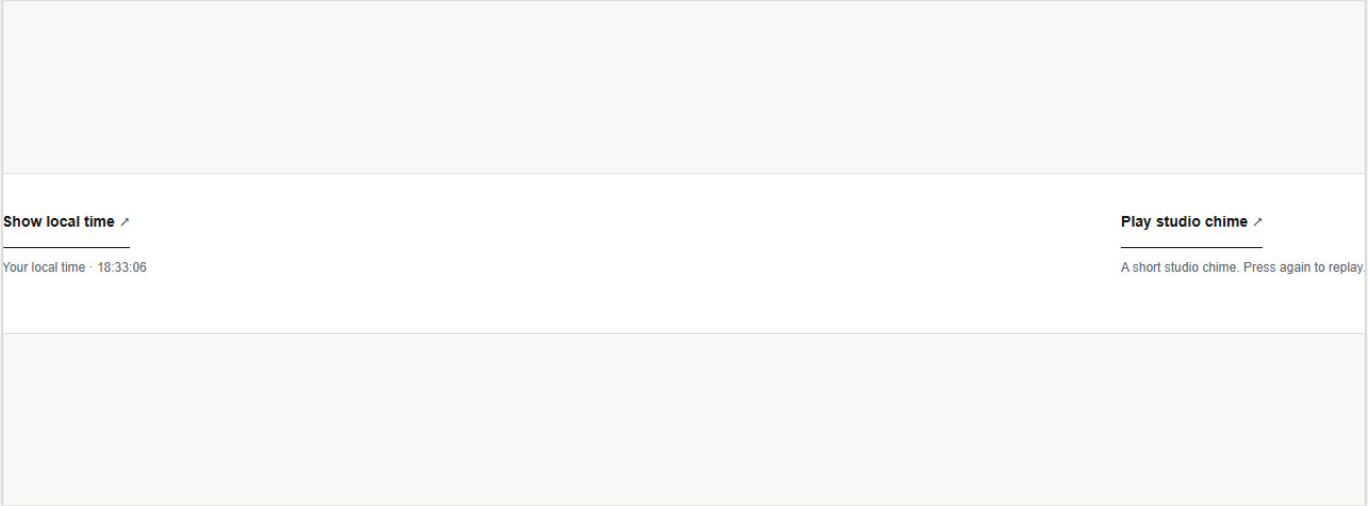
Feedback you can see and hear

Sound is triggered by deliberate user actions. The active sound is controlled to prevent overlapping playback. Gallery changes, collection filtering and form steps provide event-triggered visual feedback.

1. Create or reuse the shared audio resource and stop active playback before restarting.

2. Connect user events to gallery, collection and form transitions.

3. Respect reduced-motion preferences and provide a usable loading/fallback state.



Time and sound controls with visible user feedback.

```
js/site.js
 48    function stopSound() {
 49      notification.pause();
 50      notification.currentTime = 0;
 51      if (voice) {
 52        try {
 53          voice.stop();
js/site.js
 23    Apex.animate = (element) => {
 24      if (!element?.animate || reducedMotion.matches) return;
 25      animations.get(element)?.cancel();
 26      animations.set(
 27        element,
 28        element.animate(
 29          [
 30            { opacity: 0.4, transform: "translateY(12px)" },
 31            { opacity: 1, transform: "translateY(0)" },
 32          ],
 33          { duration: 320, easing: "cubic-bezier(.2,.7,.3,1)" },
js/inventory.js
192      if (animate) window.Apex.animate?.(grid);
```

Current source: js/site.js, js/inventory.js

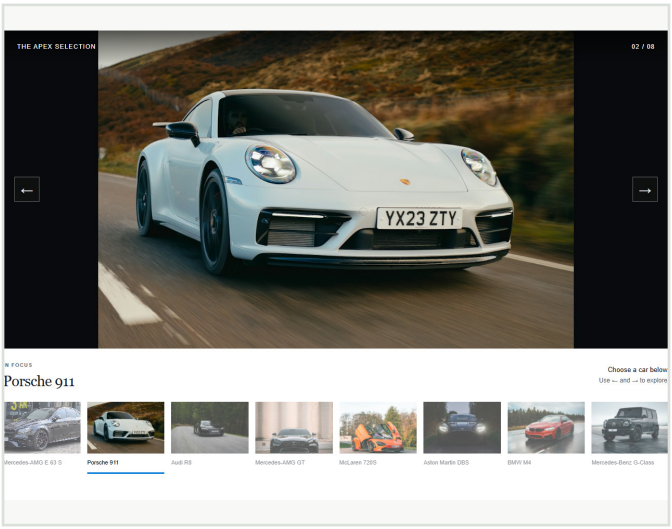
A gallery and a deeper story

The photographic showroom changes the main image when a thumbnail or arrow is selected. Read More changes additional content's visibility. These implement two allowed alternatives from the brief's content and attribute task.

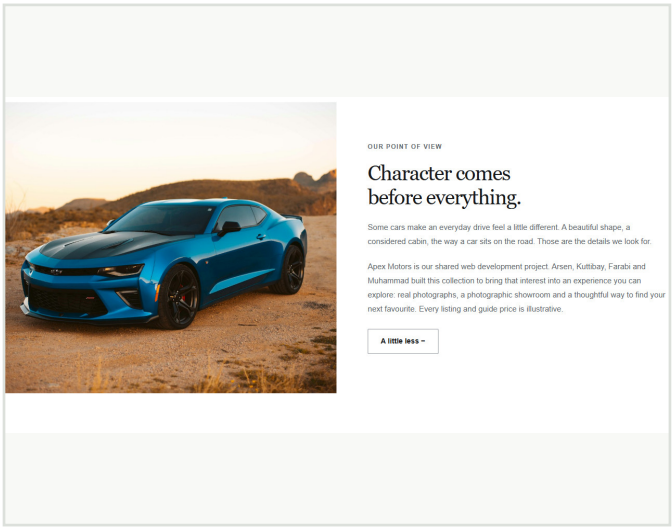
1. Select gallery controls and update the main image's src, alt and caption.

2. Keep the selected thumbnail state synchronized with Previous and Next.

3. Toggle Read More using style.display and communicate its expanded state.



Thumbnail-driven photographic showroom.



Additional story content expanded.

js/gallery.js

```
21      current = (index + thumbnails.length) % thumbnails.length;
22      const selected = thumbnails[current];
23      image.src = selected.dataset.gallerySrc;
24      image.alt =
25        selected.dataset.galleryAlt ||
26        selected.dataset.galleryTitle ||
27        "Apex Motors vehicle";
28      if (caption)
29        caption.textContent = selected.dataset.galleryTitle || image.alt;
```

js/site.js

```
359     readMoreButton.addEventListener("click", () => {
360       const expanded = readMoreButton.getAttribute("aria-expanded") !== "true";
361       readMoreContent.style.display = expanded ? "block" : "none";
362       readMoreButton.setAttribute("aria-expanded", String(expanded));
363       readMoreButton.textContent = expanded
364         ? "A little less –"
365         : "More about Apex +";
366       if (expanded) Apex.animate(readMoreContent);
367       Apex.playSound();
```

Current source: js/gallery.js, js/site.js

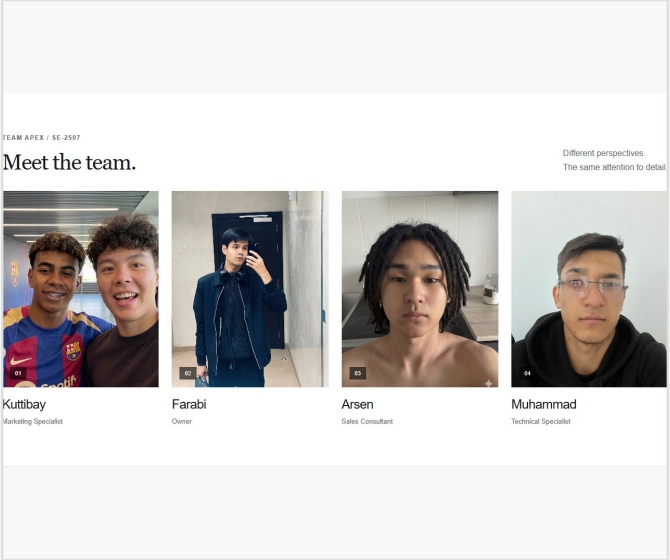
Five pages, one coherent site

Structure, appearance and behavior are separated. The five pages share branding, navigation and preferences, with local resources and the team's original identities.

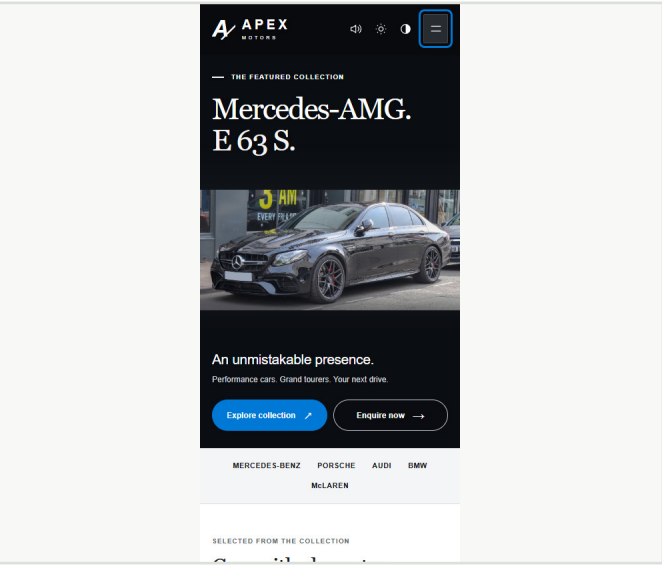
1. Link external styles and scripts; keep page-specific content in its own HTML file.

2. Use labeled controls, meaningful alternative text and visible keyboard focus.

3. Check desktop and phone layouts, navigation and the original four team portraits.



Original team and dealership theme.



Responsive phone layout.

index.html

```
14      <link rel="stylesheet" href="styles.css" />
15      <script src="js/data.js" defer></script>
16      <script src="js/site.js" defer></script>
17    </head>
18    <body data-page="index">
19      <a class="skip-link" href="#main-content">Skip to content</a>
20      <header class="site-header">
21        <div class="container navbar-inner">
22          <a class="brand" href="index.html" aria-label="Apex Motors home"
```

styles.css

```
3  :root {
4    color-scheme: light;
5    --page-bg: #fff;
6    --ink: #16181b;
7    --muted: #61666c;
8    --line: #dce0e3;
9    --surface: #f4f5f6;
10   --surface-raised: #fff;
```

Current source: index.html, styles.css

Testing and reflection

Verification. Current browser results are saved in screenshots/test-results.json. The checks cover separate-page navigation, collection filtering and saved state, appearance persistence, keyboard focus, inquiry validation and step state, local resources and responsive layouts. Use the recorded target and timestamp to identify the checked build.

Recorded result: passed: 113; total: 113

Additional runs: Direct-file browser checks 33/33.

DOM and events. Appearance, Read More and filters update selected elements. Clicks and arrows call shared event handlers and maintain visible control states.

Data and callbacks. One vehicle collection feeds cards and inquiry choices. Switch and higher-order filters process user input; the step callback separates navigation from display.

Sound and animation. User actions start controlled audio and visual transitions. Reduced-motion behavior applies to interactive transitions.

Structure and UX. Five independent pages use external CSS/JavaScript, a shared menu, local assets, labels, feedback and responsive layouts.

Publication URL:
https://vertex-blip.github.io/Car-dealership-website/assignment5-dom/
Release status. The revised source is locally verified. Committing, pushing and redeploying it are still required unless deployment.json records a successful release of this revised build. Historical live checks concern an earlier version.

Challenges and solutions. Separate pages must share vehicle data and appearance preferences, so common data and initialization are reused. Form steps retain existing inputs to preserve state. Storage failures are handled so optional persistence does not prevent interaction. The gallery reuses local photographs and keeps its main image, caption and thumbnail state synchronized.

Reflection. Reusable data and functions reduce inconsistent content. Keyboard focus, feedback and responsive testing matter as much as visual effects. The photographic showroom extends the dealership theme while the core DOM tasks remain easy to demonstrate and explain.

Submission and defense. Every member submits the PDF and ZIP through the LMS and defends the work in the practice lesson. Evaluation: DOM 10%, events 10%, data/functions 10%, sound 5%, animation 5%, structure 5%, UX 5%, defense 40%, report 10%. The deadline is the one currently shown in the LMS.